```
object WalletTimerApp extends App {

  val guardian: ActorSystem[WalletTimer.Command] =
    ActorSystem(WalletTimer(), "wallet-timer")
  guardian ! WalletTimer.Increase(1)
  guardian ! WalletTimer.Deactivate(3)

}
```

To run this app, select option `4` (`WalletTimerApp`) in the sbt shell under the same subproject. The output is as follows:

```
10:10:49,896 INFO  [WalletTimer$] - increasing to 1
10:10:52,914 INFO  [WalletTimer$] - activating       ❶
```

❶ Resumes after 3 seconds

To implement scheduling an `Activate`, you need to use the factory `Behaviors .withTimers` to create a scheduler builder. With this builder, you can use `.startSingleTimer` to create a timer that schedules a message once. These two parts are one `Behavior` that can be combined with the other `Behaviors`—`activated` and `deactivated`—just like any other.

The signature for creating a timer expects the message to be sent and the delay, as shown in the following code. `T` is the type of the actor's protocol, and `FiniteDuration` is the type of delay:

```
def startSingleTimer(msg: T, delay: FiniteDuration): Unit
```

An integer can be automatically converted to a `FiniteDuration`—for example, in seconds—by importing `scala.concurrent.duration._` and calling `.second` in the integer.